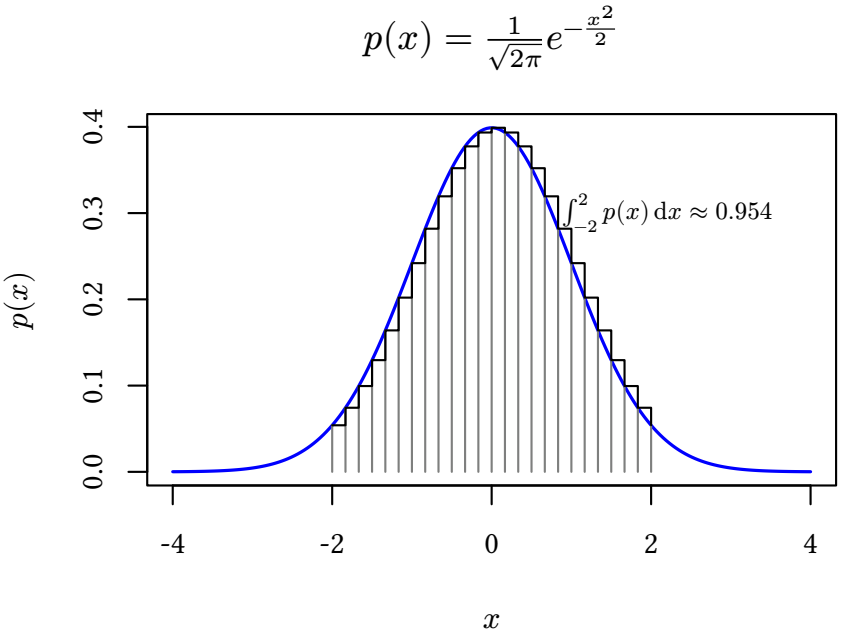
CeTZDevice

Typst Graphics for R

Andrew Junhao Wu

Version 0.5.3.1

https://github.com/SchrodingerBlume/cetzDevice



Contents

1 Introduction **1**
1.1 Acknowledgements . . . 1

I Usage and Examples **2**

2 Loading the Package **3**
2.1 Options That Affect Package Behavior . . . 3
The `metrics` and `typst` Arguments . . . 3
The `family` and `font_map` Arguments . . . 3
The `cache_dir` and `cache_key` Arguments . . . 3
The `text_mode` Argument . . . 3
The `preamble` Argument . . . 4
The `lwd_unit` Argument . . . 4
The `onefile` Argument . . . 4

3 The `cetz` Function **4**
3.1 Description . . . 4
3.2 Usage . . . 4
3.3 Font Size Calculations . . . 5
UTF-8 Output . . . 5
3.4 Examples . . . 5
Default Mode . . . 5
Raw Element Mode . . . 5
Stand Alone Mode . . . 6
Console Output Mode . . . 7
Using Typst and LaTeX Labels . . . 7
Annotating Graphics with CeTZ Commands . . . 7
Plotmath and Unicode Characters . . . 9
Using ggplot2 . . . 9
Raster Images . . . 9
Gradient Fills . . . 11
Multiple Pages . . . 11

4 The Text Measurement Cache **12**
4.1 Description . . . 12
4.2 Usage . . . 12
4.3 Examples . . . 12

II Installation Guide **13**

5 Obtaining a Typst Distribution **14**
5.1 Windows . . . 14
5.2 UNIX/Linux . . . 14
5.3 Mac OS X . . . 14
5.4 Installing CeTZ and Other Packages . . . 14
Using the Typst Package Registry . . . 14
Manual Installation . . . 14

III Package Internals **16**

6 Introduction and Background **17**

7 Anatomy of an R Graphics Device **17**
7.1 Drawing Routines . . . 17
7.2 Font Metric Routines . . . 17
7.3 Utility Routines . . . 17

8 Calculating Font Metrics **17**
8.1 Character Metrics . . . 18
8.2 Calling R Functions from C Functions . . . 18
8.3 Implementing a System Call to Typst . . . 18

9 On the Importance of Font and Style Consistency in Reports **18**
9.1 Automatic Report Generation . . . 18

References **19**

Chapter 1

Introduction

The **cetzDevice** package provides a graphics output device for R that records plots in a Typst-friendly format. The device transforms plotting commands issued by R functions into Typst code blocks. When included in a paper typeset by Typst, these blocks are interpreted with the help of CeTZ, a drawing package written by Johannes Wolf, fenjalien, and contributors. Typst handles the text in the plot as well as the text in the surrounding document. Mathematical formulae and native Typst content can therefore be used in labels and annotations.

The device works with base graphics and grid graphics. This includes plots produced by **ggplot2**, **lattice**, and **patchwork**. Figures are stored as editable source, with raster images embedded in the same file.

This document is divided into three parts. Part I describes the package functions and gives examples of their use. Part II describes the installation of Typst and CeTZ. Part III describes the implementation of the device, including its use of Typst to calculate font metrics.

1.1 Acknowledgements

This package follows the example of **tikzDevice**, originally written by Charlie Sharpsteen and Cameron Bracken and maintained by Ralf Stubner and contributors. Their work showed how an R graphics device could use a document typesetter both to measure labels and to produce the final figure. The organization of this manual and the README follows their documentation. The page layout and heading styles are adapted from their manual style file. The source project is available at https://github.com/daqana/tikzDevice.

We would like to thank Johannes Wolf, fenjalien, and the CeTZ contributors for the drawing library on which this device depends. CeTZ provides the paths, content nodes, named anchors, and canvas transformations used in the generated figures. Its source and manual are available at https://github.com/cetz-package/cetz.

We also thank the R Core Team and Paul Murrell for the R graphics engine and its documentation, the Typst developers for the typesetter, and the MiTeX developers for the interpretation of LaTeX labels.

Part I

Usage and Examples

Chapter 2

Loading the Package

The package is loaded in the usual way:

```
library(cetzDevice)
```

The main function, `cetz()`, opens a graphics device. Plotting commands then draw on that device until it is closed with `dev.off()`. The output file is written when the device closes. `cetz_save()` performs these steps for a plot object or a function containing drawing commands.

2.1 Options That Affect Package Behavior

The settings described here are arguments to `cetz()`. They apply to one device and may also be passed through `cetz_save()`.

The `metrics` and `typst` Arguments

By default, the device runs `typst query` to obtain text measurements. The `typst` argument gives the path to the executable; its default is `Sys.which("typst")`. A working Typst installation is checked before the device is opened.

The alternative `metrics = "systemfonts"` uses the **systemfonts** package. This is faster for plots containing many different labels and does not run Typst during export. Its measurements approximate Typst's text layout. This backend accepts plain text labels.

The `family` and `font_map` Arguments

The default family is Arial. The generic families `sans`, `serif`, `mono`, and `symbol` map to Arial, Times New Roman, Courier New, and Symbol. The mapping can be replaced:

```
cetz("figure.typ", family = "Libertinus Serif",
     font_map = c(sans = "Libertinus Sans",
                  serif = "Libertinus Serif",
                  mono = "DejaVu Sans Mono", symbol = "Symbol"))
plot(pressure)
dev.off()
```

Use a family installed on the machine that exports the plot and on the machine that compiles it. The **systemfonts** backend records the resolved system family in the output, including a substitute for a missing family.

The `cache_dir` and `cache_key` Arguments

Measurements are shared by devices in the same R session. Setting `cache_dir` also saves them on disk for use in later sessions. The cache distinguishes label content, font settings, backend versions, preambles, and statically named local dependencies.

Changing `cache_key` starts a separate set of measurements. This is useful after installing fonts or changing a dependency whose filename is computed by Typst code. The cache functions are described in Chapter 4.

The `text_mode` Argument

Unmarked labels are plain text by default. Setting `text_mode = "typst"` interprets them as Typst markup; `text_mode = "latex"` interprets them as LaTeX text through MiTeX 0.2.6. Both modes require `metrics = "typst"`.

The functions `cetz_typst()` and `cetz_latex()` mark individual labels. They can be used when the rest of the plot contains ordinary text. Mathematics uses dollar delimiters in both syntaxes.

The preamble Argument

The preamble is a character vector of Typst source lines inserted into both the measurement document and the output. Relative imports are resolved beside the output file. Definitions used by a Typst label must be made available in the r-label-scope dictionary:

```
cetz("preamble.typ", width = 4.5, height = 3,
     family = "Libertinus Serif", preamble = c(
       "#let term = [Shared definition]",
       "#let r-label-scope = (term: term)"))
plot(1:3, xlab = cetz_typst("#term"))
dev.off()
```

The lwd_unit Argument

The value of lwd_unit is the number of Typst points used for an R line width of one. The default is 0.75, corresponding to one ninety-sixth of an inch. Typst uses 72 points per inch.

The onefile Argument

With onefile = TRUE, every page is saved in a single module. A page is selected in Typst with r-figure(page: 2). The exported r-figures array contains the individual page functions.

With onefile = FALSE, the filename must contain one integer placeholder, for example figure-\%03d.typ. Each page is then written to its own file. Console output requires onefile = TRUE.

Chapter 3

The cetz Function

3.1 Description

The cetz() function opens an R graphics device which supports output in the CeTZ drawing language. The resulting Typst module can be imported into a document or placed inside a CeTZ canvas.

The module exports r-figure(), r-figures, r-draw(), and r-elements(). The first two produce complete plots. The latter two allow the plot to be combined with other CeTZ drawings.

3.2 Usage

```
cetz(file = "Rplots.typ", width = 7, height = 7,
     pointsize = 12, family = "Arial", bg = "white",
     standalone = FALSE, metrics = c("typst", "systemfonts"),
     typst = Sys.which("typst"),
     font_map = c(sans = "Arial", serif = "Times New Roman",
                  mono = "Courier New", symbol = "Symbol"),
     text_mode = c("plain", "typst", "latex"),
     preamble = character(), cache_dir = NULL,
     cache_key = "default", fg = "black", lwd_unit = 0.75,
     onefile = TRUE, console = FALSE)
```

The file argument gives the output path. The containing directory must exist. Width and height are in inches; pointsize is the initial text size in points. The bg and fg arguments give the starting background and foreground colors. An alpha channel may be used, including bg = "transparent".

The cetz_save() function accepts the same device arguments:

```
cetz_save("figure.typ", function() plot(pressure),
          width = 6, height = 4)
```

It also accepts a plot object which can be drawn with `print()`, such as a **ggplot2** object. The device is closed if drawing fails, and the original error is reported to the caller. On success, `cetz_save()` returns the output path invisibly; `cetz()` returns the device number invisibly.

3.3 Font Size Calculations

The device reports its dimensions to R in points, with 72 points to the inch. A label's size is obtained from the graphical parameters supplied by the graphics engine. In base graphics this includes the device point size and `cex`; grid supplies its own font size and expansion factor. The resulting size is used for both measurement and output.

Plot layout is calculated by R during export. Axis labels, margins, legends, and facets therefore depend on the selected family and size. Choose the final figure dimensions and font before drawing the plot. Changing text styles in an imported module does not run the R layout calculations again.

UTF-8 Output

The output is written as UTF-8. Ordinary labels are quoted as Typst strings, so characters such as underscores and percent signs can be used directly. Unicode characters require a font containing the corresponding glyphs. Typst's `typst fonts` command lists the families it can find.

3.4 Examples

Default Mode

The default output is a module intended for inclusion in another document. For example, the following commands produce a scatter plot with a fitted regression line:

```
cetz("default.typ", width = 4.5, height = 3,
     family = "Libertinus Serif")
plot(mtcars$wt, mtcars$mpg, xlab = "Weight", ylab = "MPG")
abline(lm(mpg ~ wt, data = mtcars), col = "red", lwd = 2)
dev.off()
```

The file is included in a Typst document as follows:

```
#import "default.typ": r-figure

#figure(
  r-figure(),
  caption: [Vehicle weight and fuel economy.],
)
```

Raw Element Mode

Every module exports `r-elements()` for drawing in an existing canvas. This corresponds to the use of `bareBones` output in **tikzDevice**. The coordinates are numeric Typst points, so the canvas length should be set to `1pt`:

```
#import "default.typ": r-elements
#import "@preview/cetz:0.5.3" as cetz

#cetz.canvas(length: 1pt, {
  import cetz.draw: *
  r-elements()
})
```

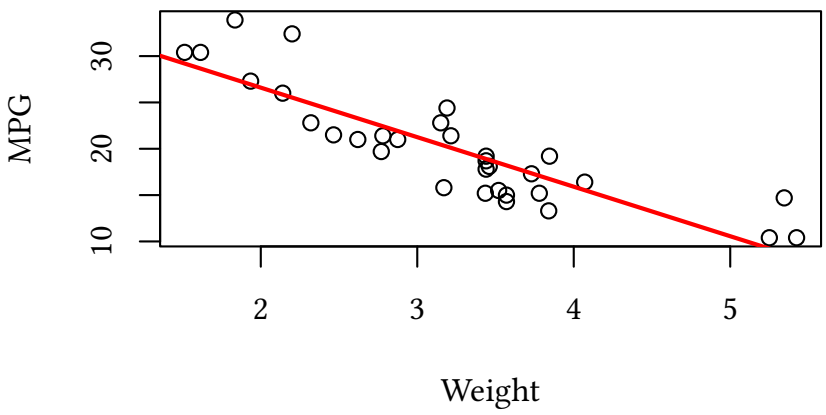


Figure 1: Output from the default mode.

The default raw elements omit R's clipping containers. Passing `clip: true` retains panel clipping, including text, images, and line widths. This mode also applies canvas transformations to the whole clipped layer. Unclipped elements follow CeTZ's native transformation rules for content and physical stroke widths.

To place the complete plot at a named position, use `r-draw()`:

```
#import "default.typ": r-draw
#import "@preview/cetz:0.5.3" as cetz

#cetz.canvas({
  import cetz.draw: *
  r-draw(origin: (0, 0), angle: 0deg, name: "plot")
})
```

This preserves the plot's physical size and clipping. Rotation, scaling, shearing, and reflection of the surrounding canvas also transform its text, strokes, images, gradients, and named anchors.

Stand Alone Mode

Setting `standalone = TRUE` adds page settings and draws all recorded pages. The resulting file can be compiled directly:

```
cetz("standalone.typ", width = 4.5, height = 3,
     family = "Libertinus Serif", standalone = TRUE)
plot(pressure, type = "b")
dev.off()
```

```
typst compile standalone.typ standalone.svg
```

For a multipage SVG export, use a page placeholder:

```
typst compile standalone.typ 'standalone-{p}.svg'
```

A stand-alone file can still be imported as a module.

Console Output Mode

With `console = TRUE`, the module is written to standard output when the device closes. An empty filename selects the same mode. The output can be captured for further processing:

```
code <- capture.output({
  cetz(file = "", width = 4, height = 3)
  plot(pressure)
  dev.off()
})
```

Using Typst and LaTeX Labels

The functions `cetz_typst()` and `cetz_latex()` mark labels for typesetting in the requested syntax. Ordinary strings in the same plot remain plain text.

```
cetz("math.typ", width = 4.5, height = 3,
     family = "Libertinus Serif")
plot(1:5, (1:5)^2,
     xlab = cetz_typst("$alpha^2$"),
     ylab = cetz_latex("$\\frac{x^2}{\\sigma}$"))
text(3, 20, cetz_typst("$integral_0^1 x^2 dif x$"))
dev.off()
```

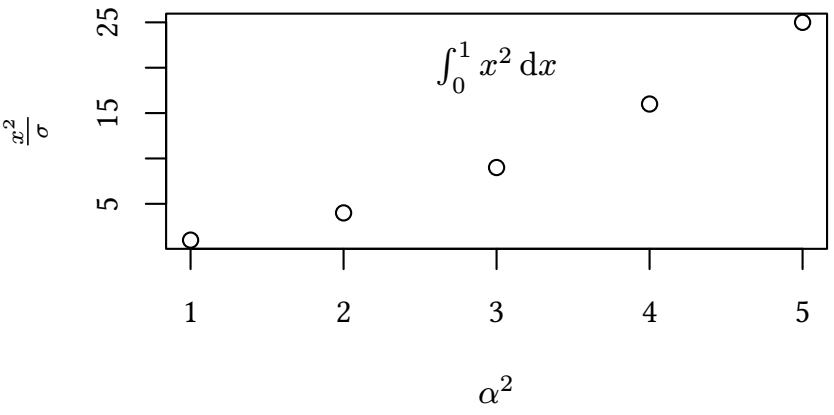


Figure 2: Typst and MiTeX mathematics in the same plot.

For plots in which every label uses one syntax, set `text_mode` at device creation. MiTeX implements a subset of LaTeX commands; it does not load LaTeX packages or execute TikZ drawings. Existing formula labels written for **tikzDevice** can be used where their commands are supported by MiTeX.

Annotating Graphics with CeTZ Commands

The `cetz_annotate()` function inserts CeTZ code into the drawing. `cetz_coord()` gives a name to an R position, and `cetz_node()` places a content node at that position. Base graphics coordinates are interpreted in the current plot's user coordinate system.

```
cetz("annotations.typ", width = 4.5, height = 3,
     family = "Libertinus Serif")
plot(1:5, (1:5)^2)
cetz_coord(4, 16, "marked")
cetz_node(2, 20, cetz_latex("$y=x^2$"), name = "label",
          options = 'frame: "rect", padding: 3')
cetz_annotate(
  'line("label.east", "marked", stroke: red, mark: (end: ">"))')
dev.off()
```

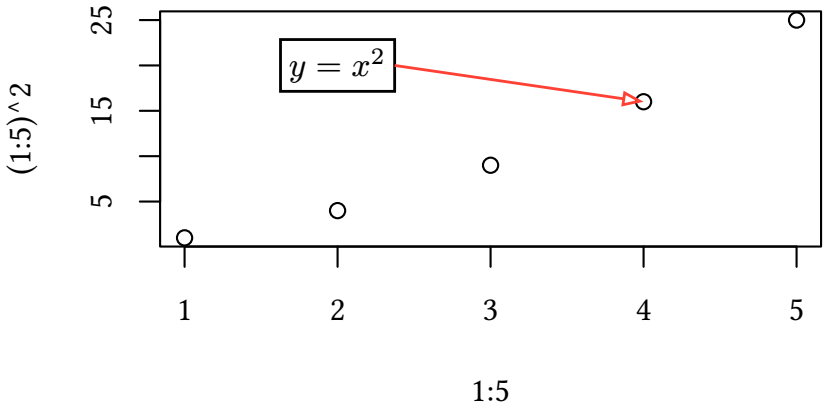


Figure 3: A CeTZ node and arrow added to a base plot.

Raw CeTZ code uses device points measured from the lower left corner of the page. Annotations are clipped to the page by default. Passing `clip = TRUE` uses the current R clipping rectangle instead. Names remain available in later clipping layers, including anchors such as `label.north`. A grouped plot exposes these as `plot.label.north` when `r-draw(name: "plot")` is used.

The grid equivalents return grobs, which are placed when drawn:

```
cetz("grid.typ", width = 4.5, height = 3,
     family = "Libertinus Serif")
grid::grid.newpage()
grid::pushViewport(grid::viewport(xscale = c(0, 5),
                                  yscale = c(0, 25)))
grid::grid.draw(cetz_coord_grob(4, 16, "marked"))
grid::grid.draw(cetz_node_grob(2, 20, "A grid label", name = "label"))
grid::grid.draw(cetz_annotate_grob(
  'line("label.east", "marked", stroke: red)'))
grid::popViewport()
dev.off()
```

Numeric grid positions use `native` units by default. Explicit unit objects can be used instead. Each node accepts one position and one text, numeric, or logical label. Its text follows the viewport rotation and graphical parameters, including color and alpha. Multiple nodes can be combined with `grid::grobTree()`.

Plotmath and Unicode Characters

The R graphics engine interprets `expression()` labels and sends the resulting character and drawing operations to the device. These labels retain R's plotmath layout. To obtain native Typst mathematics, use `cetz_typst()` instead.

```
cetz("plotmath.typ", width = 4.5, height = 3,
     family = "Libertinus Serif")
plot(1:5, main = expression(alpha^2 + beta^2 == gamma^2),
     xlab = "Observation", ylab = "Value")
dev.off()
```

Using ggplot2

A ggplot object can be passed directly to `cetz_save()`. Set the theme family to the one used in the document:

```
library(ggplot2)
p <- ggplot(mtcars, aes(wt, mpg, color = factor(cyl))) +
  geom_point() +
  labs(color = "Cylinders") +
  theme_minimal(base_family = "Libertinus Serif")
cetz_save("ggplot.typ", p, width = 4.5, height = 3,
          family = "Libertinus Serif")
```

Raster Images

The device accepts `rasterImage()`, `grid.raster()`, `geom_raster()`, and raster color bars. Images are encoded as PNG bytes in the module, so no separate image files need to accompany it. Identical images share a resource even when drawn on different pages.

```
data <- expand.grid(x = 1:20, y = 1:15)
data$value <- with(data, sin(x / 3) * cos(y / 4))
p <- ggplot(data, aes(x, y, fill = value)) +
  geom_raster() + scale_fill_viridis_c() + coord_equal() +
  theme_minimal(base_family = "Libertinus Serif")
cetz_save("raster.typ", p, width = 4.5, height = 3,
          family = "Libertinus Serif")
```

Image transparency, rotation, and reflected extents are retained. The `interpolate` flag selects Typst's `smooth` or `pixelated` image scaling.

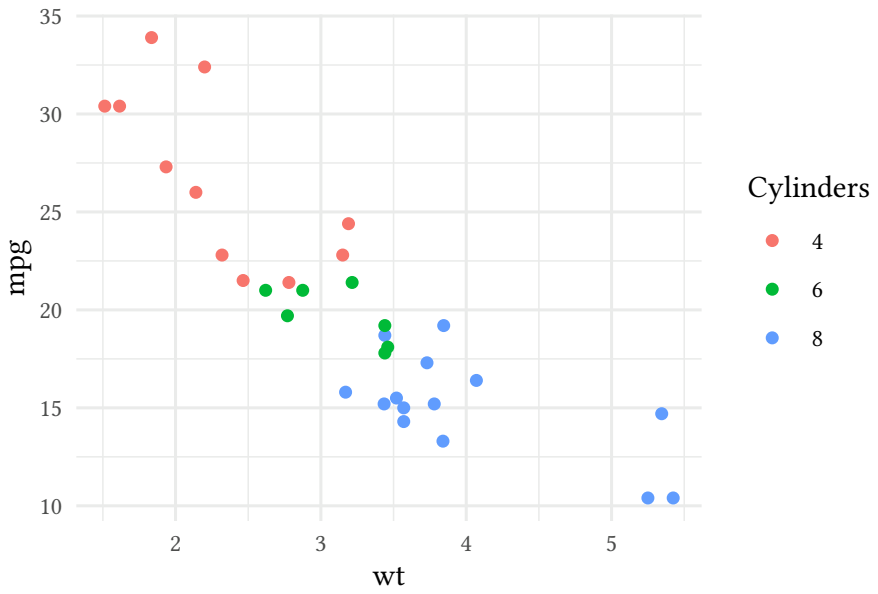


Figure 4: A ggplot2 scatter plot with a discrete legend.

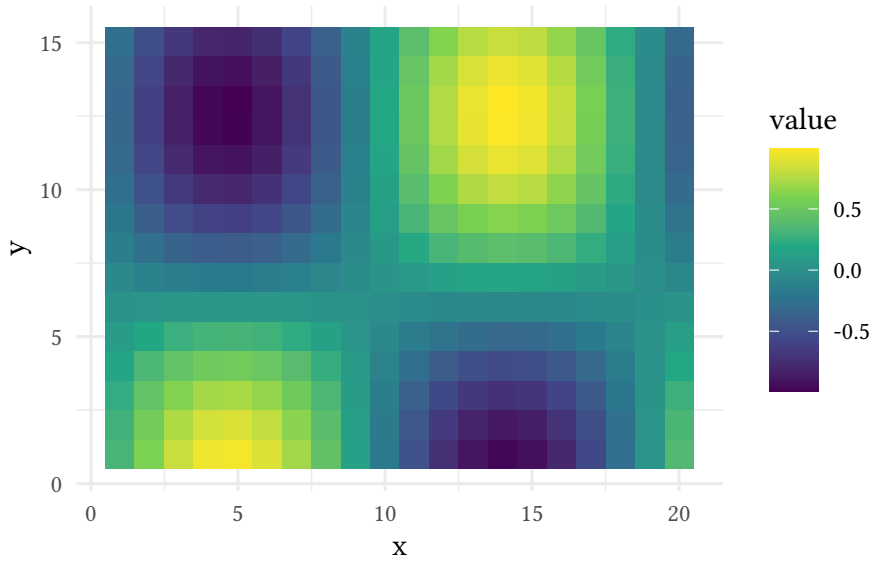


Figure 5: Embedded raster data with vector axes, labels, and legend text.

Gradient Fills

Linear gradients and radial gradients with nested circles are supported on rectangles, circles, polygons, and compound paths. Fills are embedded as SVG vectors; CeTZ draws their outlines.

```
cetz("gradient.typ", width = 4.5, height = 2.5,
     family = "Libertinus Serif")
grid::grid.newpage()
grid::grid.rect(width = 0.85, height = 0.75,
  gp = grid::gpar(col = NA, fill = grid::linearGradient(
    c("#2673b8", "#e6b450", "#d45d36"), group = FALSE)))
dev.off()
```



Figure 6: A linear gradient embedded as vector content.

The gradient retains its device-space endpoints, color stops, alpha, and `pad`, `repeat`, `reflect`, or `none` extension mode. Radial circles must have different radii, with one circle contained in the other. Reversed radii are handled by swapping the circles and color stops. Other radial geometry produces an error.

With R 4.2 or later, ggplot2 can request a vector gradient color bar:

```
p + guides(fill = guide_colourbar(display = "gradient"))
```

Tiling patterns, masks, nonrectangular clipping paths, glyph runs, and advanced compositing are not implemented.

Multiple Pages

Successive plots are recorded as separate pages. By default, all pages share one file and its image, gradient, and stroke resources. To write a file for each page:

```
cetz("page-%03d.typ", width = 4.5, height = 3,
     family = "Libertinus Serif", onefile = FALSE,
     standalone = TRUE)
plot(pressure)
hist(iris$Sepal.Length, main = "Sepal length", xlab = "Length")
dev.off()
```

Chapter 4

The Text Measurement Cache

4.1 Description

The device measures labels through the selected backend and keeps the results in a cache. Repeated labels in the same font do not require another Typst query. Cache entries contain the width, ascent, and descent in device points.

The functions `cetz_cache_info()` and `cetz_clear_cache()` inspect and clear this cache. Measurements can be requested through the usual R functions, including `strwidth()` and `strheight()`, while the device is active.

4.2 Usage

```
cetz_cache_info()
cetz_clear_cache(cache_dir = NULL)
```

The first function returns a list containing `entries`, `hits`, `misses`, and `typst_calls`. The last count includes measurement queries but not CLI version probes. The second function clears session entries and resets these counts. If a directory is supplied, it also removes the metric files belonging to this package from that directory.

4.3 Examples

```
cache <- tools::R_user_dir("cetzDevice", "cache")
cetz("figure.typ", family = "Libertinus Serif", cache_dir = cache)
plot.new()
strwidth("A measured label", units = "inches")
strwidth("A measured label", units = "inches")
dev.off()
cetz_cache_info()
```

The second request uses the cached result. Reopening a device with the same settings reuses session entries. A new R process can reuse entries stored in `cache_dir`. Damaged disk entries are recomputed.

Part II Installation Guide

Chapter 5

Obtaining a Typst Distribution

The device requires R 4.1.0 or later. The default measurement backend and the exported figures require Typst 0.15.0 or later. Typst provides a command-line executable for Windows, Linux, and macOS at https://github.com/typst/typst/releases.

After installation, the following command should print the version:

```
typst --version
```

The executable must be available to the R session. Check its location with `Sys.which("typst")`, or pass its full path to `cetz()`.

5.1 Windows

Download the Windows archive from the Typst releases page, extract it, and add the directory containing `typst.exe` to `PATH`. Restart R after changing the environment. Building the R package from source also requires Rtools for the installed version of R.

5.2 UNIX/Linux

Install Typst with the distribution's package manager or use the Linux archive from the releases page. Check the installed version, since a distribution may package an older release. A C compiler and the R development headers are needed to build **cetzDevice** from source.

5.3 Mac OS X

Typst can be installed with Homebrew:

```
brew install typst
```

The release archives also contain macOS executables. Building this package from source requires the Xcode Command Line Tools. An R session started from a graphical application may have a different `PATH` from a terminal; the `typst` argument can specify the executable directly.

5.4 Installing CeTZ and Other Packages

Using the Typst Package Registry

The generated module imports CeTZ 0.5.3 from the Typst preview registry. Typst downloads it on first compilation and stores it in the package cache. No separate CeTZ installation command is needed. LaTeX labels also import MiTeX 0.2.6.

The package version has four components. The first three identify the exact CeTZ version; the fourth counts device releases for that version, starting at one. For example, 0.5.3.2 is the second device release for CeTZ 0.5.3. The device revision restarts at one when the CeTZ version changes.

Support starts with CeTZ 0.5.3; earlier CeTZ releases have not been adapted. The function `cetz_backend()` reports the pinned CeTZ version and the minimum Typst version.

Manual Installation

For an offline machine, populate Typst's package cache with CeTZ 0.5.3, its dependencies, and MiTeX 0.2.6 if needed. Package directories and environment variables are described at https://github.com/typst/packages. Fonts used for export must also be available on the machine that compiles the document.

Install the R package from GitHub:

```r
install.packages("remotes")
remotes::install_github("SchrodingerBlume/cetzDevice")
```

To install a local source checkout, run `R CMD INSTALL .` from its root directory. The **systemfonts** package is optional and can be installed with `install.packages("systemfonts")`.

Part III

Package Internals

Chapter 6

Introduction and Background

An R plotting function does not normally write a file itself. It sends drawing operations to the graphics engine, which calls functions provided by the active device. A device can draw on a screen, write an image, or record the operations in another graphics language.

The **tikzDevice** package records those operations as TikZ source and uses LaTeX to measure text. The **cetzDevice** package takes the same approach with CeTZ and Typst. The plotting packages do not need to know which device is receiving their operations.

The C interface is implemented in `src/device.c`. It registers the device callbacks and passes drawing operations to R. Recording, measurement, and serialization are implemented in the package's `R` directory. The Typst scene code in `inst/typst/scene.typ` handles shared drawing state and placement in a canvas.

Chapter 7

Anatomy of an R Graphics Device

7.1 Drawing Routines

The device supplies callbacks for lines, polylines, polygons, rectangles, circles, compound paths, text, and rasters. It also supplies gradient callbacks. The graphics engine passes each callback the coordinates and graphical context needed to draw the object.

The C functions convert their arguments to R objects and call a recording closure. The closure stores the operations in drawing order, along with their text styles, stroke styles, and clipping state. Repeated strokes, images, and gradients share definitions in the module.

Rectangular clipping is implemented with Typst content containers. The complete drawing timeline is first resolved in a shared CeTZ context. Names and anchors created in one layer therefore remain available to later layers. The visible content of each layer is then clipped to its recorded rectangle.

7.2 Font Metric Routines

The graphics engine asks the device for string widths and character metrics while laying out a plot. The callbacks in `src/device.c` delegate these requests to the backend in `R/metrics.R`. Both measurement and output use the same font resolution and label interpretation functions.

The device passes R's plotmath requests through its character metric interface. Typst and MiTeX labels are measured as complete pieces of content instead.

7.3 Utility Routines

The device reports its size and resolution to the engine and records page changes. Closing the device finishes the final page, serializes the module, and removes temporary measurement files. The output is assembled before an existing file is replaced.

Drawing and measurement callbacks use unwind cleanup on R 4.6 and later to release a device lock after a callback error. `cetz_save()` lets the drawing call finish unwinding before closing the device and reporting the error.

Chapter 8

Calculating Font Metrics

Text measurements determine the positions of labels and the space used by margins and legends. The Typst backend writes a small document using the requested label, family, face, and point size. It calls `typst query` and reads the

numeric dimensions from a metadata element. The same text expression is later written into the figure.

8.1 Character Metrics

The graphics engine requires a width, ascent, and descent for individual characters. Typst measures a box using the text's bounds. Two zero-width probe boxes, placed on opposite sides of the baseline, separate the measured height into ascent and descent. The results are returned in device points.

8.2 Calling R Functions from C Functions

The device stores references to an R recording closure and an R measurement closure. These are preserved while the device is open. Drawing callbacks build a call containing the operation type, geometry, and graphical context. Measurement callbacks pass the label and text style. The closures share the state of that particular device.

The `close` callback releases the preserved objects after writing the output. Errors during final output are stored in the device state and reported by `cetz_save()`.

8.3 Implementing a System Call to Typst

The measurement document is created beside the output file so that relative preamble imports resolve in the same directory. The executable is called through `system2()`, with paths quoted separately. The query selects one metadata label and returns its JSON value.

Cache keys contain the backend version, platform, text expression, preamble, relevant Typst environment variables, and local dependency digests. Static local imports are followed recursively. Disk entries are written through temporary files and are checked when read.

Chapter 9

On the Importance of Font and Style Consistency in Reports

The font used in a plot affects its appearance and its layout. A label measured in one family and drawn in another can extend past an axis or overlap a neighboring label. The default backend asks Typst to measure the same content that will appear in the figure.

Set the device family to the document's text family and use that family in plotting themes. A document may use several font sizes, but a plot should be exported at the size at which it will be read. Scaling the whole plot also scales its text and strokes.

9.1 Automatic Report Generation

An analysis script can produce the figure modules before the report is compiled. The Typst document then imports the modules just as it imports other source files. A persistent measurement cache avoids repeating queries for unchanged labels on later runs.

```
Rscript analysis.R
typst compile report.typ
```

The output records graphics operations. Data mappings, statistical transformations, and axis layout remain in the R analysis, where they can be rerun when the data or figure dimensions change.

References

tikzDevice Charlie Sharpsteen, Cameron Bracken, Ralf Stubner, and contributors. *tikzDevice: R Graphics Output in LaTeX Format*.
https://github.com/daqana/tikzDevice

CeTZ Johannes Wolf, fenjalien, and contributors. *CeTZ: Drawing with Typst*.
https://github.com/cetz-package/cetz

Typst *Typst Documentation*.
https://typst.app/docs/

MiTeX *MiTeX: LaTeX Support for Typst*.
https://github.com/mitex-rs/mitex

R R Core Team. *R Internals*, section "Graphics Devices".
https://cran.r-project.org/doc/manuals/r-release/R-ints.html